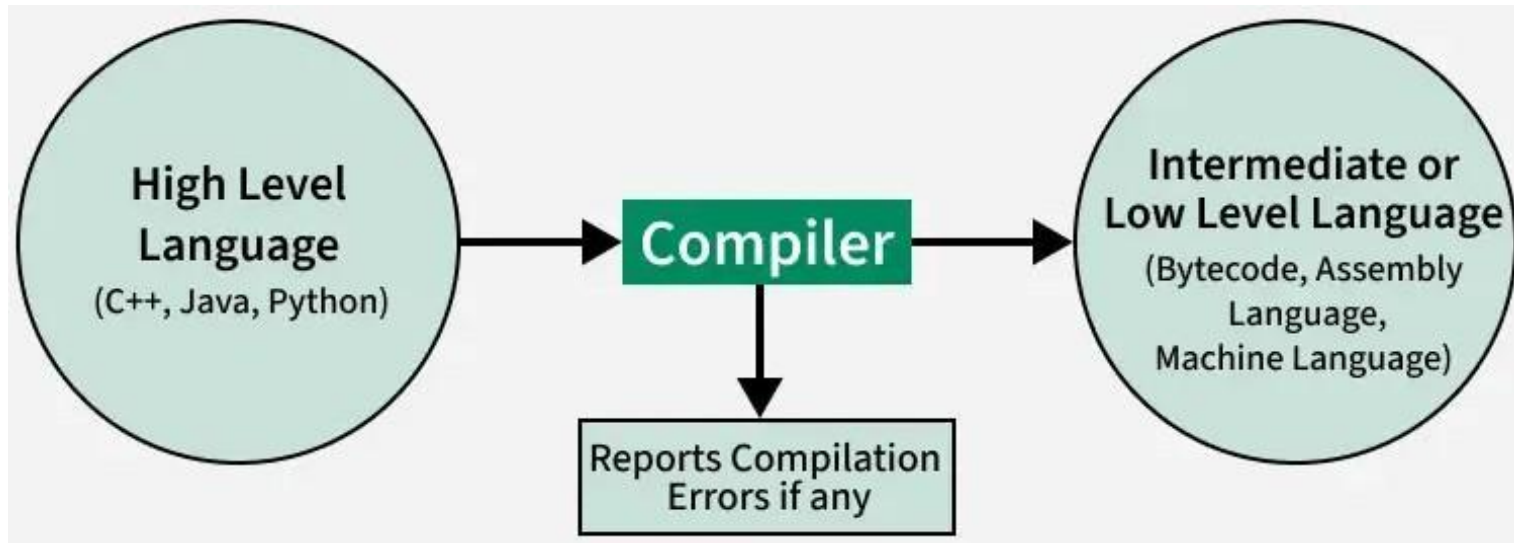


UNIT-1

INTRODUCTION

Dr Sandeep Sahu
Associate Professor, LNCT, Bhopal



➤ History of Compilers

- ✓ In the 1950s, Grace Hopper developed the first compiler, leading to languages like FORTRAN (1957), LISP (1958), and COBOL (1959).
- ✓ The 1960s saw innovations like ALGOL, and the 1970s introduced C and Pascal. Modern compilers focus on optimization, supporting object-oriented features and Just-in-Time compilation.
- ✓ Compilers have revolutionized programming, enabled complex systems and improved software efficiency.

Need for Compiler

Machine language:

- Media that directly communicate with computer
- Error prone

Assembly Language:

- Use mnemonic name for operation codes and data address
- Translator assembler helps it
- Still it required how specific computer operates

High level programming:

- Express algorithms in more natural notation
- Without knowing how specific computer works
- Translator compiler helps it

Assembler

- ❖ Translates assembly language program into machine language
- ❖ Uses mnemonic codes for operation codes and data addresses.

Preprocessor

- ❖ A translator whose source language and object language are high level language is termed as preprocessor.
- ❖ C Preprocessor-begins with #

Macro processor

- ❖ A system program that replaces the macro calls in an assembly language program by the macro definition.
- ❖ During this, formal arguments are converted into the actual arguments.

History of compilers

- Early computers written in Assembly language
- 1950 - Machine –independent programming language were introduced - leads to the development of compilers
- 1952 - first compiler was written by **Grace Hopper** for A-0 Programming languages
 - Translation is viewed as the compilation of a sequence of machine language subprograms selected from library
- 1957 - first complete compiler introduced by FORTRAN team led by **John Backus** at IBM
- 1960 – COBOL compiled on multiple architecture

- 1984 – **GNU C Compiler by Richard Stallman**
 - For Unix Operating system
- 1987 – first release of GCC compiler
 - First portable ANSI C compiler as free software
- 1992 – version 2.0
 - Ability to compile C++
- 1997 – compiler EGCS was created –
 - to improve optimization
 - 2001 – version 3.0

History of compilers

- 1962 – first self-hosting compiler – compiling its own source code by **Tim Hart and Mike Levin** at MIT for Lisp
- 1970 – Pascal and C Compilers written in their own languages
 - Building self-hosting compiler creates bootstrapping problem – avoid by compiling either by a compiler in different languages or running the compiler in different languages

History of compilers

- 1995 – java compiler was introduced
- Major java compilers
 - **javac**, included in JDK - open-sourced since nov 2006.
 - **Gcj** - a part of gcc which compiles C, Fortran, Pascal and other programming languages .
 - **ECJ** - the Eclipse Compiler for Java, is an open source used by the Eclipse JDT based on IBM Visual Age's Java compiler.
 -

STRUCTURE OF A COMPILER FRONT-END AND BACK-END OF COMPILER

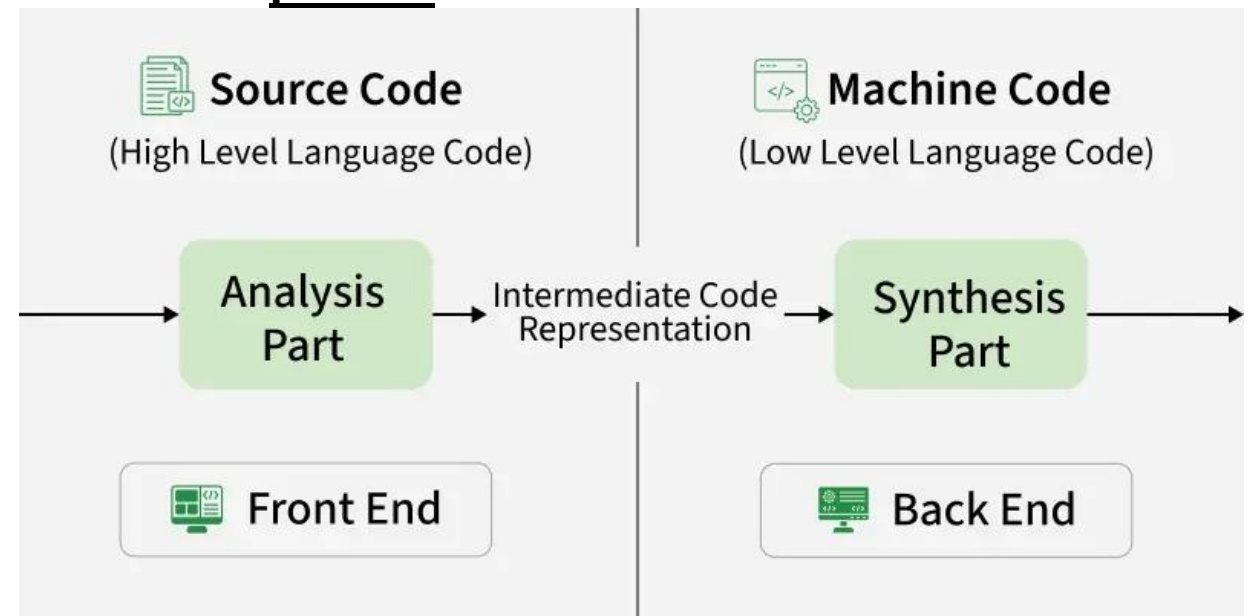
➤ **Phases of a compiler:** A compiler operates in phases. A phase is a logically interrelated operation that takes source program in one representation and produces output in another representation. The phases of a compiler are shown in below

There are two phases of compilation.

a. Analysis (Machine Independent/Language Dependent): The analysis phase breaks the source program into its basic components and creates an intermediate representation of the program. It is sometimes referred to as front end.

b. Synthesis (Machine Dependent/Language independent): The synthesis phase creates the final target program from the intermediate representation. It is sometimes referred to as back end.

Compilation process is partitioned into no-of-sub processes called **phases**.



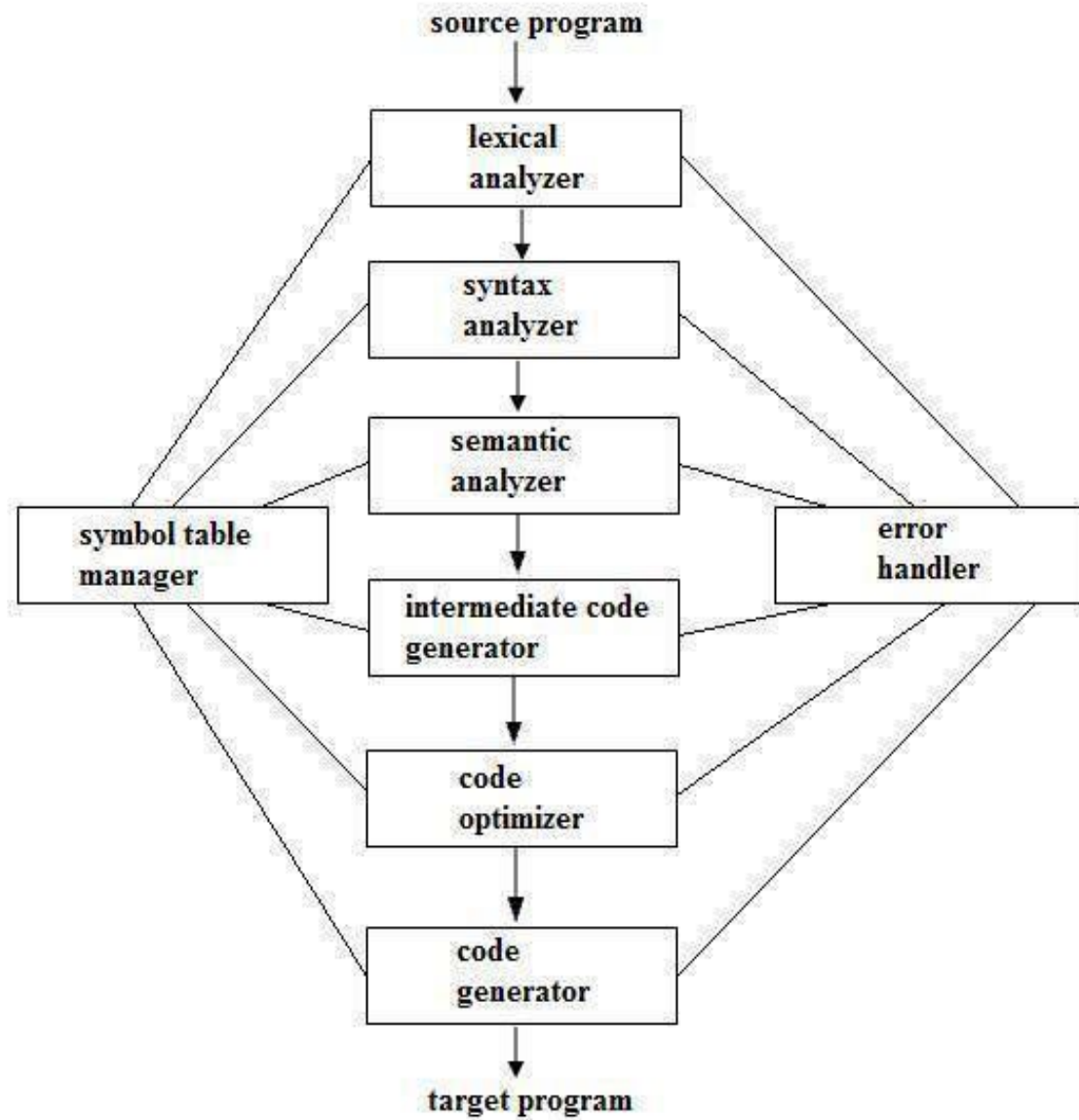
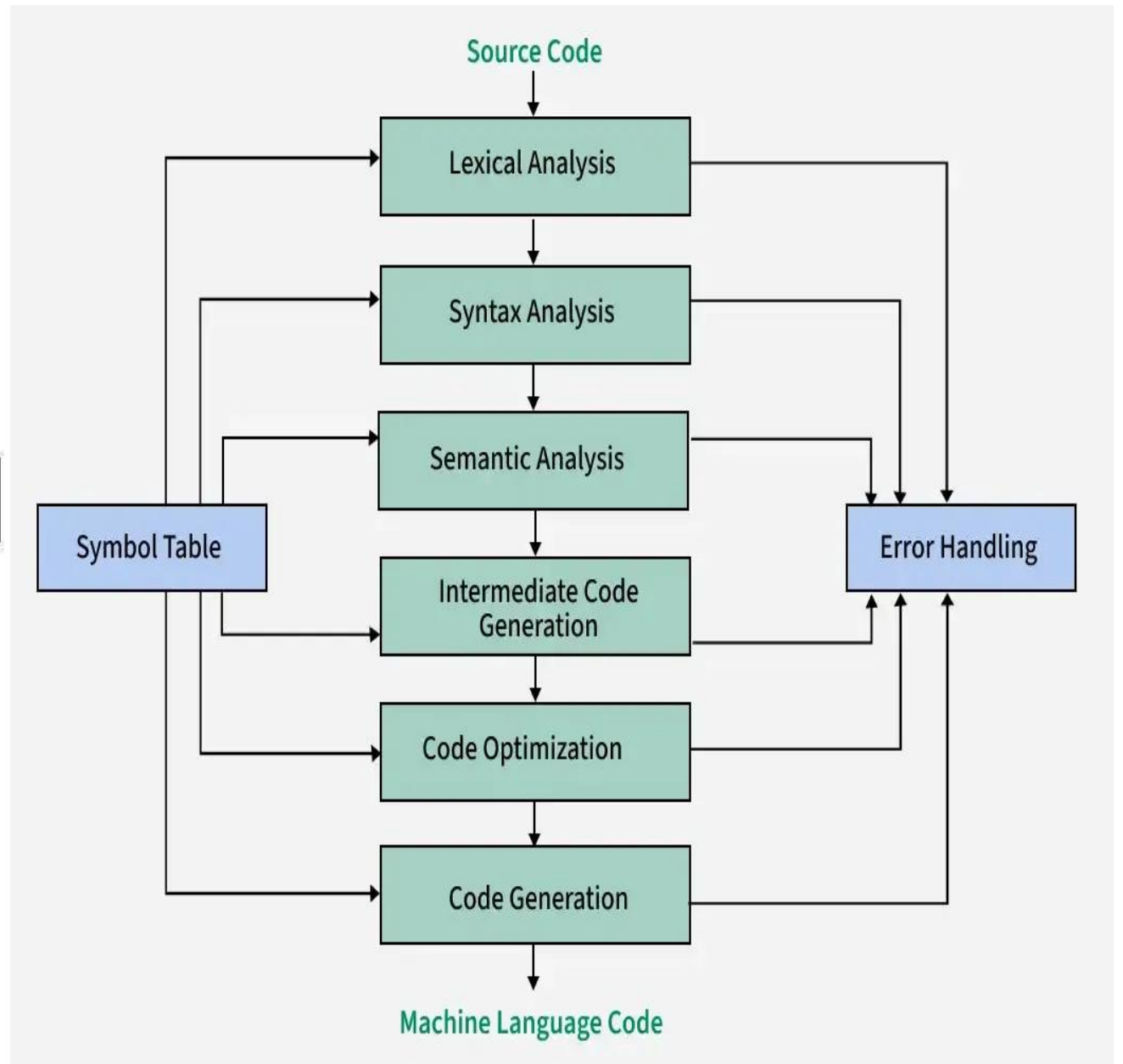
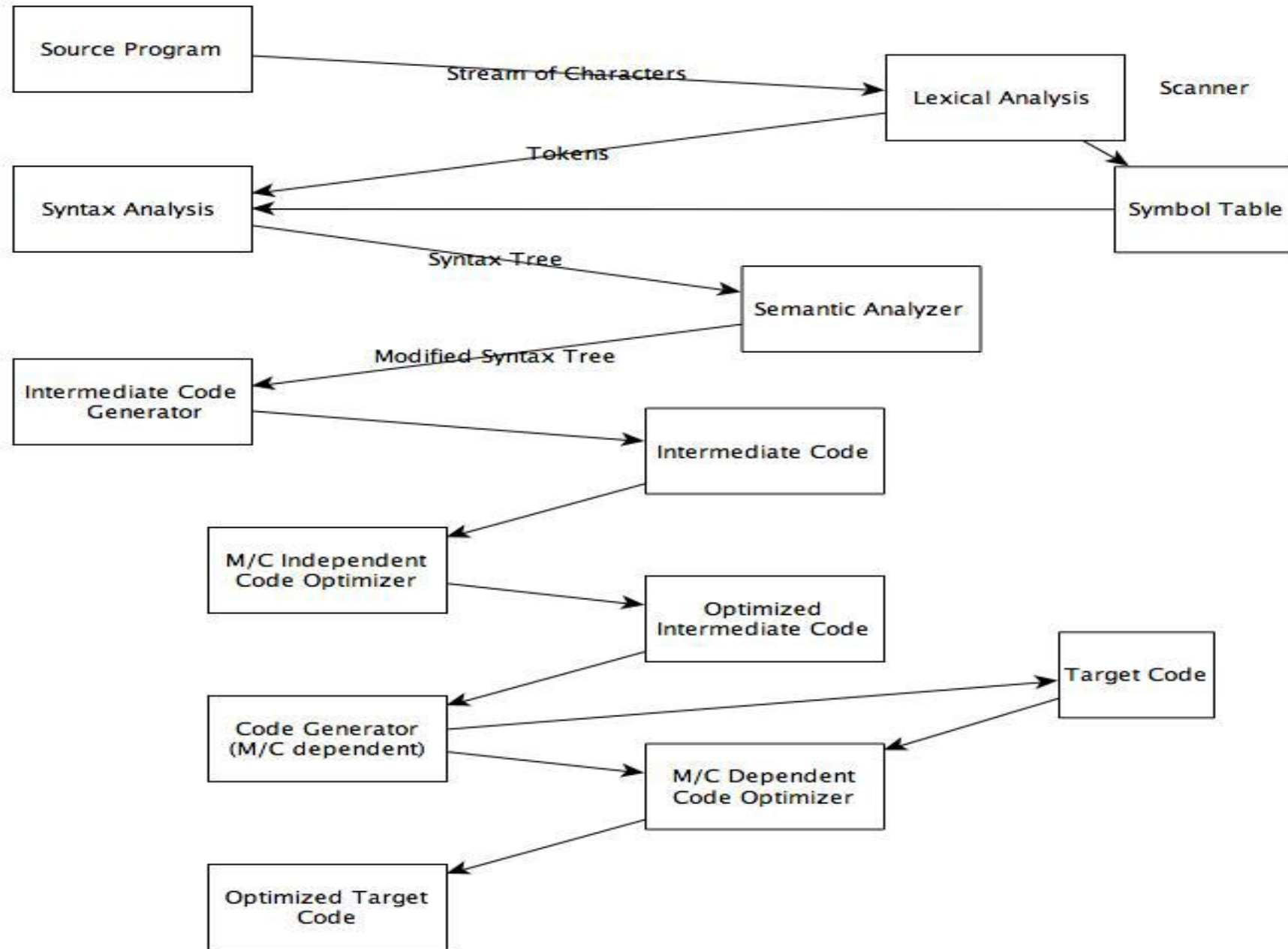


Fig 1.5 Phases of a compiler





Phases of a compiler

1. Lexical Analysis:-

The source program one character at a time, carving the source program into a sequence of automatic units called **tokens**.

2. Syntax Analysis:-

The second stage of translation is called Syntax analysis or parsing. In this phase expressions, statements, declarations etc. are identified by using the results of lexical analysis. Syntax analysis is aided by using techniques based on formal grammar of the programming language.

3. Semantic Analysis: Verifies the meaning of the code and ensures it follows language semantics. **Example:** Reports undeclared variables or type mismatches (e.g., adding a string to an integer).

4. Intermediate Code Generations:-

An intermediate representation of the final machine language code is produced.

This phase bridges the analysis and synthesis phases of translation.

5. Code Optimization :-

This is optional phase described to improve the intermediate code so that the output runs faster and takes less space.

6. Code Generation:-

The last phase of translation is code generation. A number of optimizations to **reduce the length of machine language program** are carried out during this phase. The output of the code generator is the machine language program of the specified computer.

MAJOR DATA STRUCTURE IN COMPILER

- The major data structures used in a compiler are essential for managing and transforming source code into machine code. The primary ones include the symbol table, parse tree (or abstract syntax tree), and stack.
- **Symbol Table:** This is a fundamental data structure used throughout all phases of a compiler to store information about identifiers (such as variable names, function names, classes, etc.) and their attributes (like type, scope, and memory location). It is often implemented using a hash table for efficient lookups and insertions.
- **Parse Tree (or Syntax Tree):** Generated during the syntax analysis (parsing) phase, a parse tree is a hierarchical representation of the program's grammatical structure based on the language's formal grammar. It is later transformed into a more compact Abstract Syntax Tree (AST) which is used by later phases like semantic analysis, optimization, and code generation.
- **Stack:** Stacks are used extensively in the parsing phase (especially for bottom-up parsers) and during runtime environment management. They help in handling function calls, local variable storage via activation records, and evaluating expressions due to their Last-In-First-Out (LIFO) property.
- **Literal Table:** This data structure stores constant values and string literals found in the source code.
- **Arrays and Linked Lists:** These are often used as building blocks for implementing the more complex data structures mentioned above, such as for storing the elements of a symbol table or managing linked scopes.
- **Intermediate Representation (IR):** While not a single, simple data structure, the compiler uses various forms of intermediate representations (e.g., three-address code) to bridge the gap between the source code and the target machine code, which can be optimized in the middle-end of the compiler.

LIST OF COMPILERS AND TYPES

LIST OF COMPILERS

1. Ada compilers
2. ALGOL compilers
3. BASIC compilers
4. C# compilers
5. C compilers
6. C++ compilers
7. COBOL compilers
8. Common Lisp compilers
9. ECMAScript interpreters
10. Fortran compilers
11. Java compilers
12. Pascal compilers
13. PL/I compilers
14. Python compilers
15. Smalltalk compilers

Types of Compilers

- **Native/Self Compiler:** Produces code for the same machine and OS it runs on.
- **Cross Compiler:** Runs on one platform but produces executable code for a different platform.
- **Source-to-Source (Transpiler):** Translates high-level language code to another high-level language (e.g., C++ to C).
- **Just-In-Time (JIT) Compiler:** Compiles bytecode into native machine code at runtime (e.g., JVM, [V8 JavaScript engine](#)).
- **Multi-pass Compiler:** Processes source code through multiple, modular phases to optimize output.
- **Single-pass Compiler:** Performs all compilation steps in one pass.

Other Compiler Classifications

- **Interpreter:** Executes code directly without generating machine code first (e.g., Python).
- **Assembly Compiler (Assembler):** Converts assembly language into machine code.
- **Incremental Compiler:** Recompiles only the parts of the code that have changed.

Common Compilers by Language

- **C/C++:** GCC, Clang, Intel C++ Compiler, Microsoft Visual C++
- **Java:** Javac (OpenJDK), Eclipse Compiler for Java (ECJ)
- **C#:** Roslyn, [Mono gmcs](#)
- **Go:** gc (go)
- **Pascal:** Free Pascal (FPC)
- **Ada:** GNAT

Types of Compilers

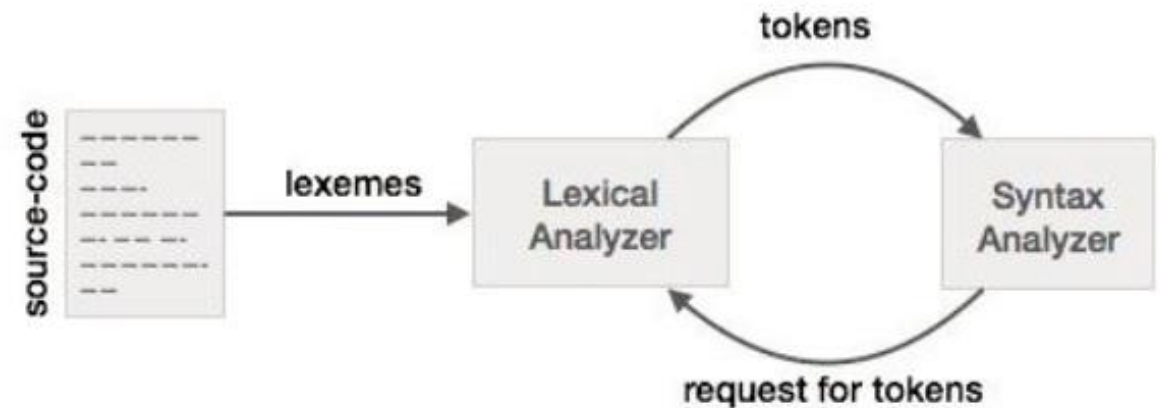
- Self Compiler:** When the compiler runs on the same machine and produces machine code for the same machine on which it is running then it is called as self compiler or resident compiler.
- Cross Compiler:** The compiler may run on one machine and produce the machine codes for other computers then in that case it is called a cross-compiler. It is capable of creating code for a platform other than the one on which the compiler is running.
- Source-to-Source Compiler:** A Source-to-Source Compiler or transcompiler or transpiler is a compiler that translates source code written in one programming language into the source code of another programming language.
- Single Pass Compiler:** When all the phases of the compiler are present inside a single module, it is simply called a single-pass compiler. It performs the work of converting source code to machine code.
- Two Pass Compiler:** Two-pass compiler is a compiler in which the program is translated twice, once from the front end and the back from the back end known as Two Pass Compiler.
- Multi-Pass Compiler:** When several intermediate codes are created in a program and a syntax tree is processed many times, it is called Multi-Pass Compiler. It breaks codes into smaller programs.
- Incremental Compiler:** It compiles only the parts of the code that have changed, rather than recompiling the entire program. This makes the compilation process faster and more efficient, especially during development.

Lexical analysis (Scanner): Input buffering, Specification & Recognition of Tokens

- The lexical analyzer (or "lexer" or "tokenizer" or "scanner") is the **first phase of a compiler** that reads the source code character by character and groups them into meaningful units called **tokens**.
- Its structure often uses finite automata to define and recognize patterns, and it works closely with the parser in the compilation process.

❖ Tokens

- Lexemes are said to be a sequence of characters (alphanumeric) in a token.
- There are some predefined rules for every lexeme to be identified as a valid token.
- These rules are defined by grammar rules, by means of a pattern.
- A pattern explains what can be a token, and these patterns are defined by means of regular expressions.
- In programming language, keywords, constants, identifiers, strings, numbers, operators and punctuations symbols can be considered as tokens.



➤ Specifications of Tokens:

- ✓ Alphabets/Identifiers
- ✓ Constants (literals)
- ✓ Separators
- ✓ Keywords
- ✓ Strings
- ✓ Special characters or symbols

- The lexical analyzer and the parser work in a producer-consumer relationship.
- The parser makes calls to the lexical analyzer (e.g., a "get next token" command) whenever it needs the next token.
- The **lexer** then scans the source code until the next token is identified, and returns it to the parser.
- This division of labor simplifies the design of the parser, allowing it to focus solely on the grammatical structure of the program.
- A lexical analyzer, or **lexer**, works by reading source code character by character and converting the sequence of characters into a stream of meaningful units called **tokens**.
- This process is often called scanning or tokenization.

For example, for the C/C++ code: `int sum = 0;`

The lexical analyzer would produce a stream of tokens like:

- <keyword (KW), "int">
- <identifier (ID), "sum"> (with a pointer to its symbol table entry)
- <operator (OP), "="
- <literal, "0">
- <punctuation, ";">

➤ Structure of a Lexical Analyzer

- ❖ The lexical analyzer is not a single, monolithic block but interacts with other components of the compiler, primarily the parser and the symbol table.
- **Input Buffering:** The analyzer often uses a specialized input buffer (sometimes a pair of buffers with sentinels) to efficiently read the source code characters, which helps in scenarios requiring "lookahead" (reading one or more characters ahead to decide the current token type).
- **Scanner:** This core component is generally implemented using a **Finite Automaton (FA)**, specifically a Deterministic Finite Automaton (DFA), which is derived from regular expressions that define the patterns of valid tokens in the programming language.
- **Token Output:** Upon identifying a complete sequence of characters (a *lexeme*) that matches a pattern, the scanner generates a token (a [token name, optional attribute value] pair).
- **Symbol Table Management:** The lexical analyzer interacts with the symbol table to store information about identifiers and constants, returning a pointer to the relevant symbol table entry as the token's attribute.

❖ Working of a Lexical Analyzer

➤ The lexical analyzer operates on demand from the next phase of the compiler, the parser. The general working mechanism is as follows:

1.Parser Request: The parser sends a "get next token" request to the lexical analyzer.

2.Character Reading: The analyzer reads the input source code character by character from left to right, using its input buffers.

3.Pattern Matching: It groups the sequence of characters into a lexeme by matching them against the predefined patterns (regular expressions) for tokens (e.g., keywords, identifiers, operators, numeric constants, punctuation). The **longest match rule** is applied when a sequence of characters could potentially match multiple patterns.

4.Whitespace and Comment Removal: The analyzer typically ignores and strips out whitespace, comments, and new-line characters, as these are not needed for the subsequent syntax analysis phase.

5.Token Generation: Once a valid lexeme is identified, the analyzer generates a token and returns it to the parser. The token often includes an integer code for its type and an optional value or pointer to the symbol table.

6.Error Handling: If an invalid character sequence (one that doesn't match any valid token pattern) is encountered, the analyzer generates a lexical error message, often correlating it with the line number in the source code.

➤ This process simplifies the parser's task by providing a clean, structured stream of tokens rather than raw characters. Tools like **Lex** and **Flex** automate the creation of a lexical analyzer by taking regular expressions as input and generating the corresponding C code for the scanner.

- Recognize tokens and ignore white spaces, comments

i	f		(	x	1		*	x	2	<	1	.	0	)	{
---	---	--	---	---	---	--	---	---	---	---	---	---	---	---	---

- Generates token stream

if	(	x1	*	x2	<	1.0	)	{
----	---	----	---	----	---	-----	---	---

➤ Approaches to implementation

- ☐ Use assembly language Most efficient but most difficult to implement
- ☐ Use high level languages like C Efficient but difficult to implement
- ☐ Use tools like lex, flex Easy to implement but not as efficient as the first two cases

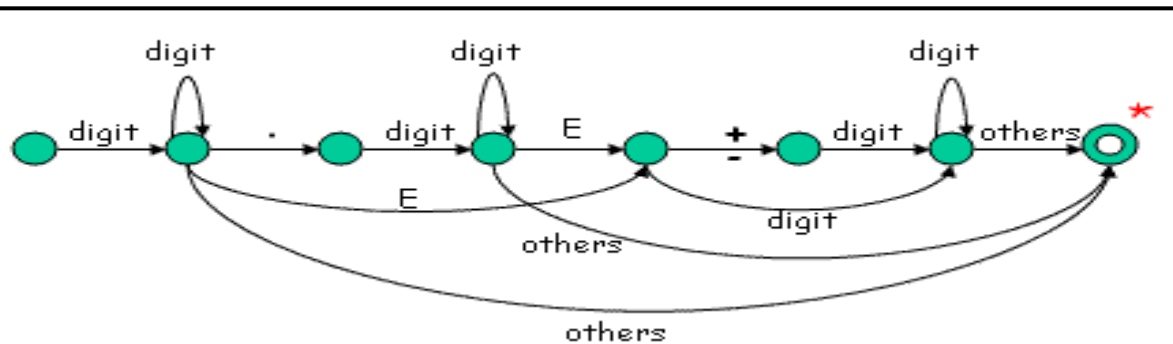
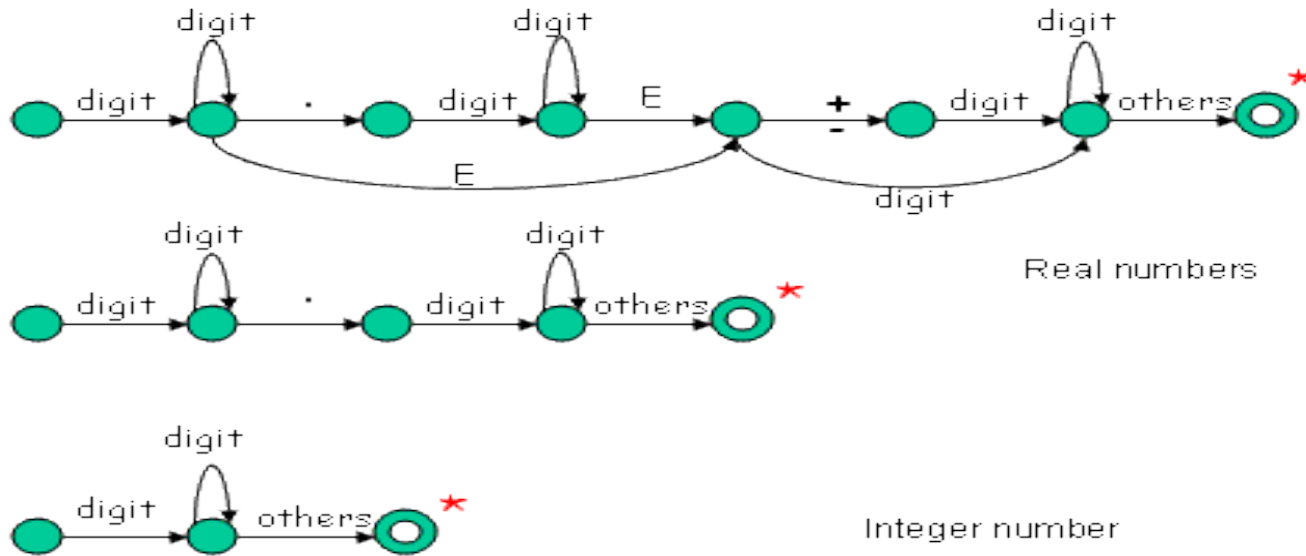
➤ Lexical analyzers can be implemented using many approaches/techniques:

- ✓ **Assembly language** : We have to take input and read it character by character. So, we need to have control over low level I/O. Assembly language is the best option for that because it is the most efficient. This implementation produces very efficient lexical analyzers. However, it is most difficult to implement, debug and maintain.
- ✓ **High level language like C**: Here we will have a reasonable control over I/O because of high-level constructs. This approach is efficient but still difficult to implement.
- ✓ **Tools like Lexical Generators and Parsers**: This approach is very easy to implement, only specifications of the lexical analyzer or parser need to be written. The lex tool produces the corresponding C code. But this approach is not very efficient which can sometimes be an issue. We can also use a hybrid approach wherein we use high level languages or efficient tools to produce the basic code and if there are some hot-spots (some functions are a bottleneck) then they can be replaced by fast and efficient assembly language routines.

EXAMPLE: RULES OF IDENTIFIERS DECLARATIONS IN C/C++

- The rules for naming identifiers (such as for variables, functions, and arrays) in the C/C++ programming language are designed to ensure the compiler can uniquely recognize each element.
- The key rules are:
- Allowed Characters: Identifiers can only contain letters (A-Z, a-z), digits (0-9), and the underscore (_) character.
- First Character: The name must start with a letter (A-Z, a-z) or an underscore (_). It cannot start with a digit.
- No Keywords: You cannot use any of the C reserved keywords (e.g., int, return, if, while) as an identifier name.
- No Whitespace: Blank spaces, commas, or any other special characters (like @, \$, %, -) are not allowed within the identifier.
- Case Sensitive: C is a case-sensitive language, meaning that name and Name are considered two different, unique identifiers.
- Uniqueness of Name: Each identifier must be unique within its specific scope (e.g., within a function or globally declared).
- Length: While there is no strict limit defined by the language standard, most compilers support at least 31 significant characters in an identifier name. It is best practice to keep names concise and meaningful.

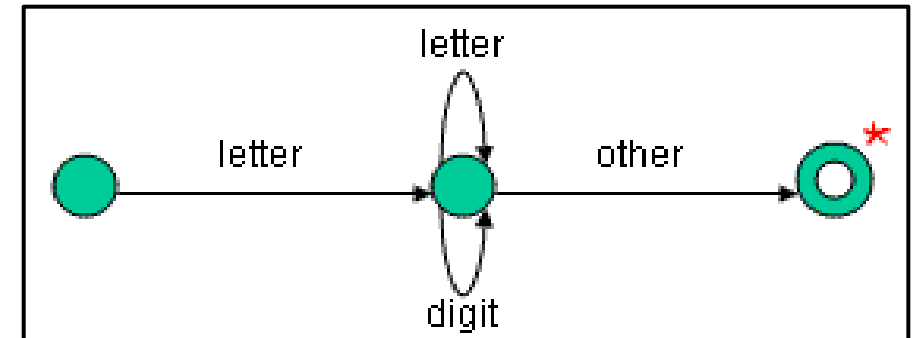
- Transition diagram for **Unsigned Numbers** : We can have *three* kinds of unsigned numbers and hence need three transition diagrams which distinguish each of them.
- The first one recognizes *exponential* numbers.
- The second one recognizes *real* numbers. The third one recognizes *integers* .



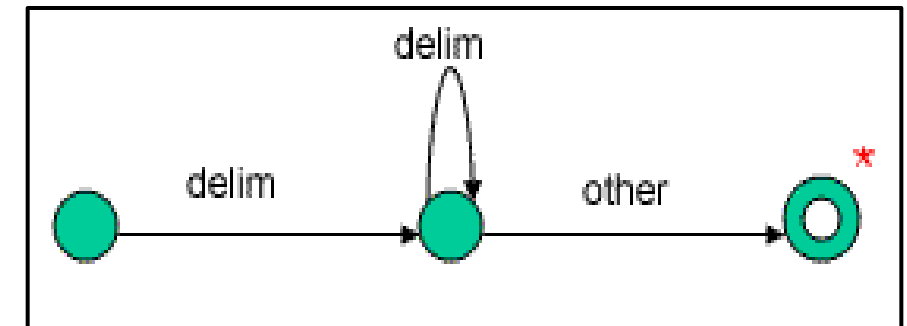
Regular Expression:

identifiers $\rightarrow$ letter (letter | digit)*

Transition diagram for identifier



Transition diagram for white spaces



<-- Another transition diagram for unsigned numbers

Design of Lexical Analyzer Generator

- A lexical analyser generator (e.g., Lex/Flex) is designed by converting regular expression patterns defining tokens into a combined Non-deterministic Finite Automaton (NFA).
- This NFA is then converted to a Deterministic Finite Automaton (DFA) using subset construction, which is optimized into a transition table to create a fast, efficient lexical scanner.

❖ Key Components in the Design of a Lexical Analyzer Generator:

Input Specification: The user provides a file (e.g., .l) containing regular expression patterns and corresponding actions (C code).

- **NFA Construction**: The generator parses regular expressions and converts each into an NFA, which are then combined into a single NFA with a new start state.
- **DFA Conversion**: The combined NFA is converted into a DFA using the subset construction algorithm to ensure efficiency, as NFAs can have multiple paths for one input.
- **Transition Table Generation**: The DFA is represented as a transition table (state vs. input character), allowing the scanner to determine the next state quickly.
- **Lexer Simulator**: The generator produces a C program (lex.yy.c) containing a function, typically yylex(), which simulates the DFA on an input stream, matching tokens and executing corresponding actions.

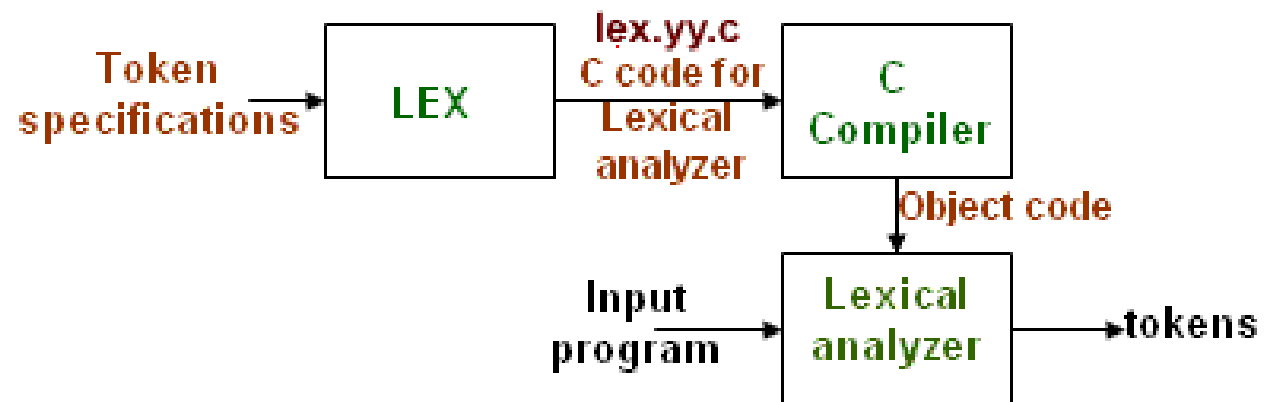
Workflow:

Input: Regular expressions + Actions --> Lex Compiler.

Process: Converts RegEx --> NFA --> DFA --> Transition Table.

Output: lex.yy.c (Scanner) --> C Compiler --> Executable Scanner.

- In this section, we consider the design of a software tool that automatically constructs the lexical analyzer code from the LEX specifications.
- LEX is one such lexical analyzer generator which produces C code based on the token specifications. This tool has been widely used to specify lexical analyzers for a variety of languages.
- We refer to the tool as Lex Compiler, and to its input specification as the Lex language.
- Lex is generally used in the manner depicted in the slide.
- First, a specification of a lexical analyzer is prepared by creating a program lex.l in the lex language.
- Then, the lex.l is run through the Lex compiler to produce a C program lex.yy.c .
- The program lex.yy.c consists of a tabular representation of a transition diagram constructed from the regular expressions of the lex.l, together with a standard routine that uses the table to recognize lexemes.
- The actions associated with the regular expressions in lex.l are pieces of C code and are carried over directly to lex.yy.c.
- Finally, lex.yy.c is run through the C compiler to produce an object program a.out which is the lexical analyzer that transforms the input stream into a sequence of tokens.



Key Design Principles:

- Lookahead:** Supports lookahead operators to resolve ambiguity in token matching.
- Prioritization:** If multiple patterns match, the rule defined first in the specification takes precedence.
- Efficiency:** Uses DFAs to ensure the scanning process is fast and efficient.
- Tokenization:** Converts character streams into structured tokens, ignoring whitespaces and comments.

LEX

- In compiler design, **Lex** is a computer program (tool for creating) that acts as a **lexical analyzer generator**.
- It takes a high-level specification of token patterns using regular expressions and produces source code (typically in C) for a lexical analyzer, also known as a "scanner" or "tokenizer".
- The primary function of this generated analyzer is to read a raw input stream of characters (the source code) and convert it into a sequence of meaningful units called **tokens**.

❖ Function and Role

- Tokenization:** Lex breaks the input text into tokens, which are the fundamental building blocks of a program (e.g., keywords, identifiers, operators, constants, punctuation).
- Whitespace and Comment Removal:** The generated lexer automatically removes unnecessary characters like spaces, tabs, newlines, and comments from the source code, simplifying subsequent compiler phases.
- Error Detection:** It can detect and report basic lexical errors, such as invalid characters or unmatched strings, along with the line number.
- Integration with Yacc:** Lex is commonly used with Yacc (Yet Another Compiler Compiler), a parser generator. Lex feeds the stream of tokens to the parser generated by Yacc, which then checks for the grammatical structure of the program.

The Phases of a Compiler

Phase	Output	Sample
Programmer	Source string	C=A+B ;
Scanner (performs lexical analysis)	Token string	\C', \=', \A', \+', \B', \;' And <i>symbol table</i> for identifiers
Parser (performs syntax analysis based on the grammar of the programming language)	Parse tree or abstract syntax tree	<pre> C = / \ C + / \ A B </pre>
Semantic analyzer (type checking, etc)	Parse tree or abstract syntax tree	C.int, A.int, B.int
Intermediate code generator	Three-address code, quads, or RTL	C=A+B-D t1=t2+t3 t4= t1-t5
Optimizer	Three-address code, quads, or RTL	Optimized Code
Code generator	Assembly code	ADD r1,r2 MOV C,r1
Peephole optimizer	Assembly code	Optimized Code

Summary: What is a compiler?

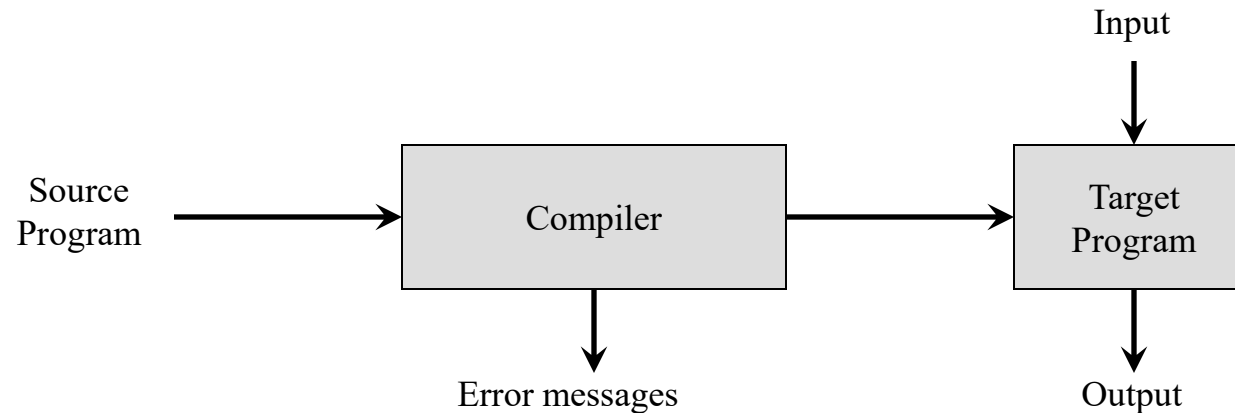
- A compiler is a program translates (or compiles) a program written in a **high-level** programming language (the source language) that is suitable for human programmers into the **low-level** machine language (target language) that is required by computers.
- During this process, the compiler will also attempt to spot and report obvious programmer mistakes that detect during the translation process.

Why we use high-level language for programming?

- Using a high-level language for programming has a large impact on how fast programs can be developed. The main reasons for this are:
 1. Compared to machine language, the notation used by programming languages is closer to the way humans think about problems.
 2. The compiler can spot some obvious programming mistakes.
 3. Programs written in a high-level language tend to be shorter than equivalent programs written in machine language.
 4. The same program can be compiled to many different machine languages and, hence, be brought to run on many different machines.

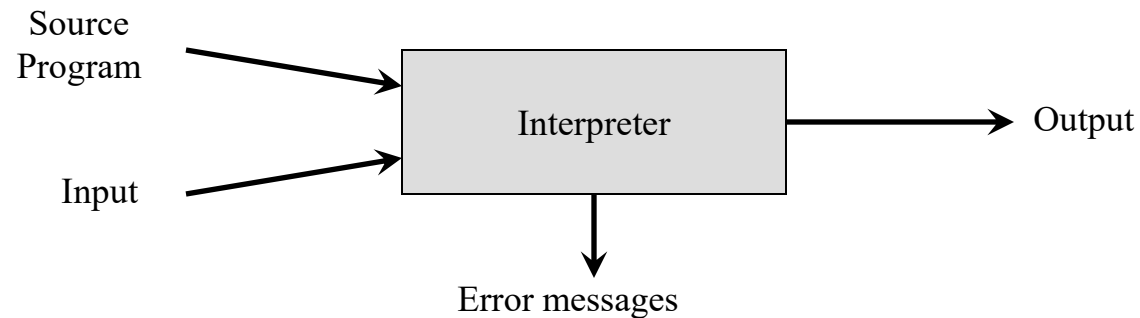
Compilers and Interpreters

- “Compilation”
 - Translation of a program written in a source language into a semantically equivalent program written in a target language



Compilers and Interpreters (cont'd)

- **“Interpretation”**
 - Performing the operations implied by the source program



Compiler vs. Interpreter

Compiler

- Takes Entire program as input
- It is **Faster**
- intermediate object code is **generated**.
- Required **more** memory Due to intermediate object code
- Program **not need compile** every time
- **Errors** are displayed after **entire program** is checked.
- **Debugging** is comparatively **hard**.
- Ex: C, C++.

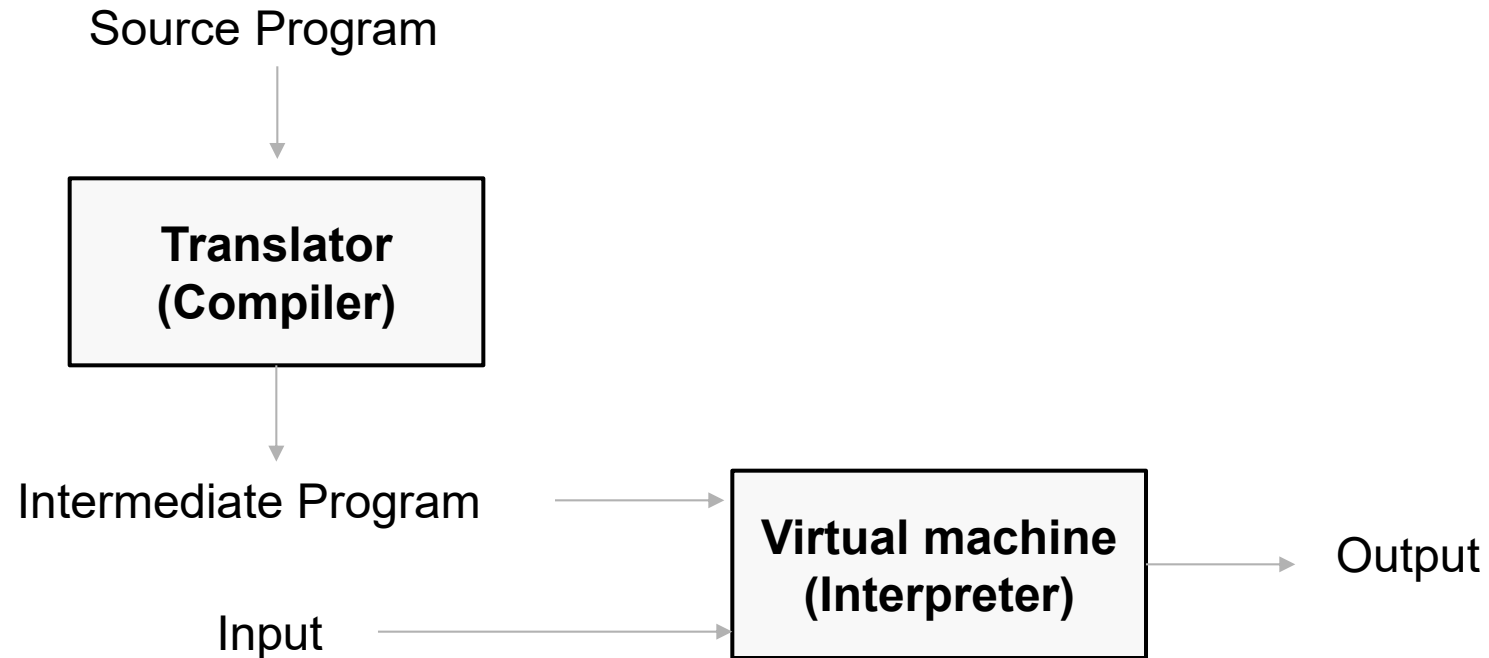
Interpreter

- Take single instruction as input
- It is **Slower**
- **No** intermediate code is **generated**
- Required **less** memory As no intermediate code is generated
- **Every time** higher level program is converted into lower-level program.
- **Errors** are displayed **for every instruction** interpreted.
- **Debugging** is **easy**.
- Ex: python, Ruby, basic.

Hybrid Compiler

- Compilation and interpretation may be combined to implement a programming language.
- The compiler may produce intermediate-level code which is then interpreted rather than compiled to machine code.

Ex: java



The Analysis-Synthesis Model of Compilation

- **There are two parts to compilation:**
 - **Analysis** determines the operations implied by the source program which are recorded in a tree structure
 - **Synthesis** takes the tree structure and translates the operations therein into the target program

Other Tools that Use the Analysis-Synthesis Model

- Editors (syntax highlighting)
- Pretty printers (e.g. doxygen)
- Static checkers (e.g. lint and splint)
- Interpreters
- Text formatters (e.g. TeX and LaTeX)
- Silicon compilers (e.g. VHDL)
- Query interpreters/compilers (Databases)

Compiler-Construction Tools

1. **Parser generators** that automatically produce syntax analyzers from a grammatical description of a programming language.
2. **Scanner generators** that produce lexical analyzers from a regular-expression description of the tokens of a language.
3. **Syntax-directed translation engines** that produce collections of routines for walking a parse tree and generating intermediate code.
4. **Code-generator generators** that produce a code generator from a collection of rules for translating each operation of the intermediate language into the machine language for a target machine.
5. **Data-flow analysis engines** that facilitate the gathering of information about how values are transmitted from one part of a program to each other part. Data-flow analysis is a key part of code optimization.
6. **Compiler-construction toolkits** that provide an integrated set of routines for constructing various phases of a compiler.

The Evolution of Programming Language

Classification by generation: -

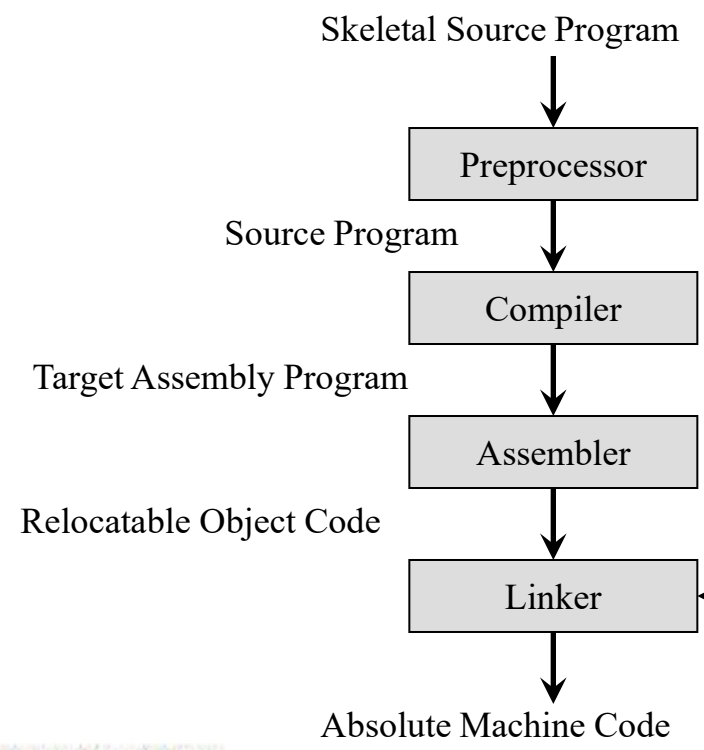
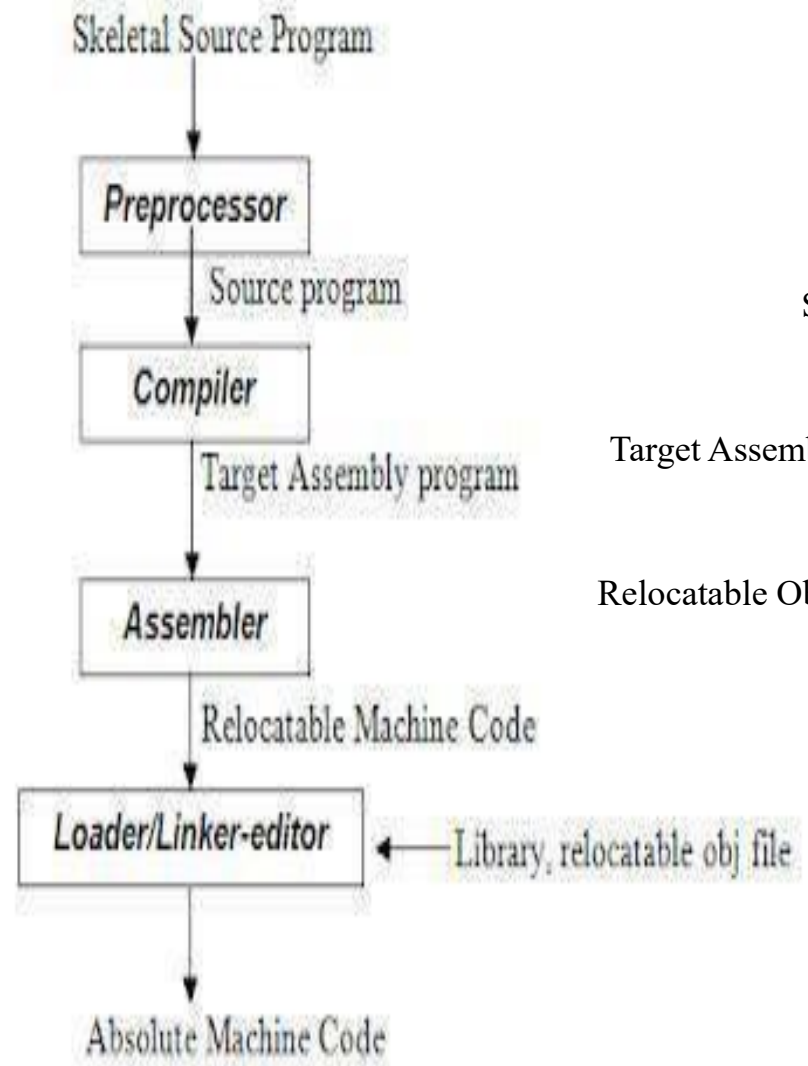
- ❑ **First-generation languages:** machine languages
- ❑ **Second-generation :** assembly languages
- ❑ **Third-generation :** higher-level languages like Fortran, Cobol, Lisp, C, C++, C#, and Java.
- ❑ **Fourth-generation languages:** languages designed for specific applications like NOMAD for report generation, SQL for database queries, and Postscript for text formatting.
- ❑ **Fifth-generation language** has been applied to logic- and constraint-based languages like Prolog and OPS5.

Impacts on Compilers

- The advances in programming languages placed new demands on compiler writers.
- Compiler writers would take maximal advantage of the new hardware capabilities.
- Good software-engineering techniques are essential for creating and evolving modern language processors.

The Grouping of Phases

- **Compiler** front and back ends:
 - Analysis (*machine independent* front end)
 - Synthesis (*machine dependent* back end)
- **Passes**
 - A collection of phases may be repeated only once (*single pass*) or multiple times (*multi pass*)
 - Single pass: usually requires everything to be defined before being used in source program
 - Multi pass: compiler may have to keep entire program representation in memory



Libraries and Relocatable Object Files

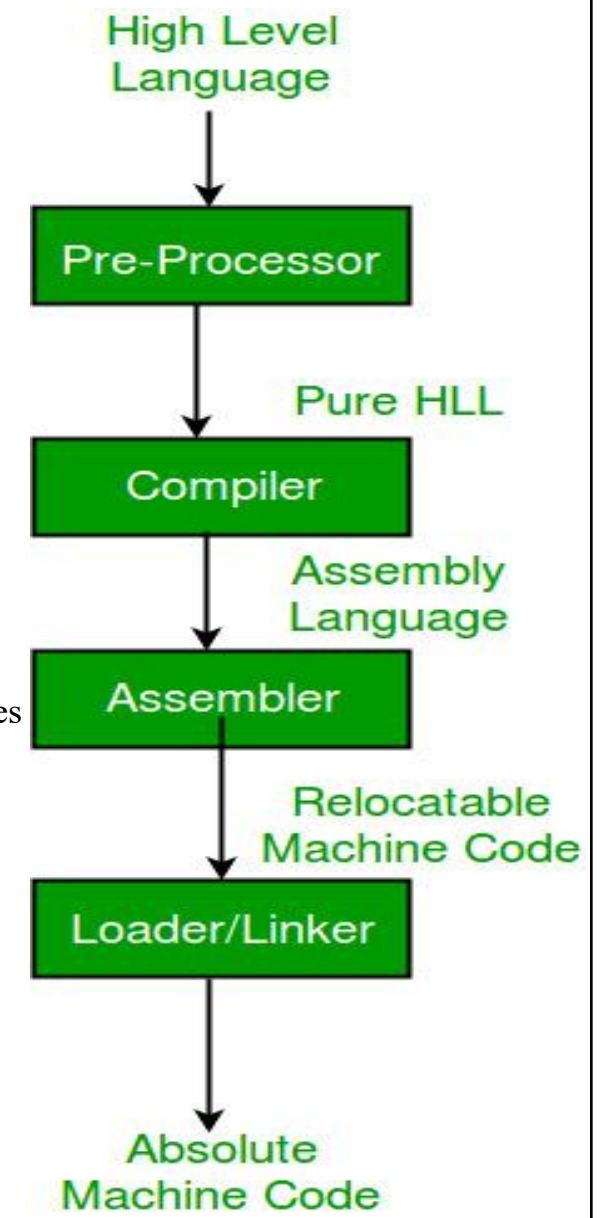


Fig 1.1 Language-processing System

❑ Preprocessor

➤ A preprocessor produces input to compilers. They may perform the following functions.

1. Macro processing: A preprocessor may allow a user to define macros that are short hands for longer constructs.
2. File inclusion: A preprocessor may include header files into the program text.
3. Rational preprocessor: these preprocessors augment older languages with more modern flow-of-control and data structuring facilities.
4. Language Extensions: These preprocessor attempts to add capabilities to the language by certain amounts to build-in macro

❑ Assembler

➤ Programmers found it difficult to write or read programs in machine language.

➤ They begin to use a mnemonic (symbols) for each machine instruction, which they would subsequently translate into machine language. Such a mnemonic machine language is now called an assembly language.

➤ Programs known as assembler were written to automate the translation of assembly language into machine language.

➤ The input to an assembler program is called source program, the output is a machine language translation (object prog).

❑ Loader and Link-editor

➤ Once the assembler produces an object program, that program must be placed into memory and executed.

➤ The assembler could place the object program directly in memory and transfer control to it, thereby causing the machine language program to be executed.

➤ This would waste core by leaving the assembler in memory while the user's program was being executed.

➤ Also, the programmer would have to retranslate his program with each execution, thus wasting translation time.

➤ To overcome these problems of wasted translation time and memory. System programmers developed another component called loader

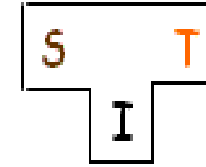
Why learn about compilers?

- It is considered a topic that you should know in order to be “well-cultured” in computer science.
- A good craftsman should know his tools, and compilers are important tools for programmers and computer scientists.
- The techniques used for constructing a compiler are useful for other purposes as well.
- There is a good chance that a programmer or computer scientist will need to write a compiler or interpreter for a domain-specific language.

Bootstrapping

- A compiler can be characterized by three languages: the source language (S), the target language (T), and the implementation language (I)
- The three language S, I, and T can be quite different. Such a compiler is called cross-compiler

- This is represented by a T-diagram as:

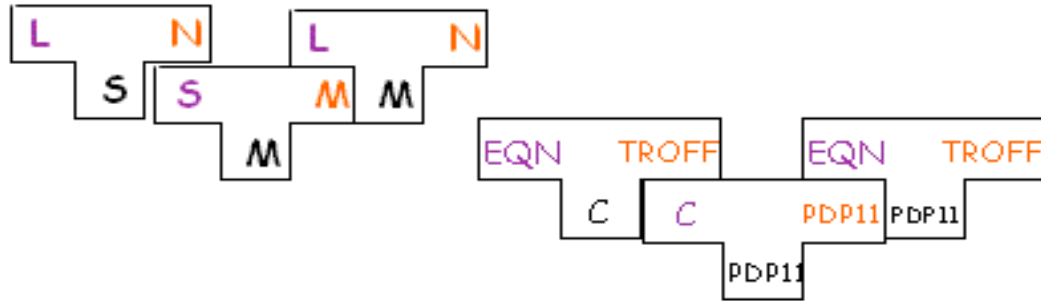


- In textual form this can be represented as

$S_I T$

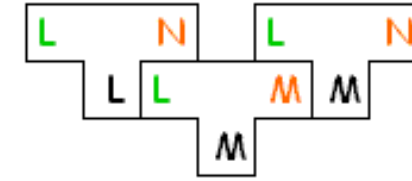
- Bootstrapping is a fairly common practice when creating a programming language.
- Many compilers for many programming languages are bootstrapped, including compilers for ALGOL, BASIC, C, Common Lisp, D, Eiffel, Elixir, Factor, Go, Haskell, Java, Modula-2, Nim, Oberon, OCaml, Pascal, PL/I, Python, Rust, Scala, Scheme, TypeScript, Vala, Zig and more.
- Using bootstrapping not only lets us understand and use it but also opens the way for a multitude of language innovations and compiler optimization, taking us beyond programming languages and software tools as we know it today.
- **Compilers are of two kinds: *native* and *cross*.**
- **Native compilers are written in the same language as the target language.**
- **For example, SMM is a compiler for the language S that is in a language that runs on machine M and generates output code that runs on machine M.**
- **Cross compilers are written in different language as the target language. For example, SNM is a compiler for the language S that is in a language that runs on machine N and generates output code that runs on machine M.**

- Write a cross compiler for a language **L** in implementation language **S** to generate code for machine **N**
- Existing compiler for **S** runs on a different machine **M** and generates code for **M**
- When Compiler **L_SN** is run through **S_MM** we get compiler **L_MN**

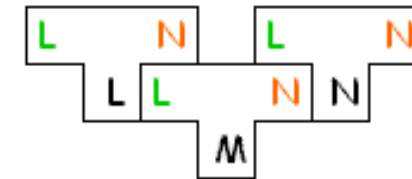


- The compiler of **L_SN** is written in language **S**.
- This compiler code is compiled once on **S_MM** to generate the compiler's code in a language that runs on machine **M**.
- So, in effect, we get a compiler that converts code in language **L** to code that runs on machine **N** and the compiler itself is in language **M**.
- In other words, we get **L_MN**.

- Suppose **L_LN** is to be developed on a machine **M** where **L_MM** is available



- Compile **L_LN** second time using the generated compiler



**Bootstrapping a Compiler:
the Complete picture**

